

Aufgabe 1: Analyse der Linearen Suche

Implementieren Sie den Algorithmus der linearen Suche in Java, messen Sie seine Laufzeit im Worst Case für verschiedene Eingabegrößen N und belegen Sie empirisch seine Klassifizierung als $O(n)$.

Teil 1: Implementierung und Messumgebung

A. Implementierung des Algorithmus

1. Implementieren Sie in Java die Methode `linearSearch(int[] a, int val)`.
 - Die Methode führt die lineare Suche im Array `a` durch, um den Wert `val` zu finden.
 - Rückgabe:
 - Der Index des gefundenen Wertes.
 - Wenn der Wert nicht gefunden wird, geben Sie `-1` zurück.
2. Implementieren Sie einen Java-Client, der ein sehr großes Array (`int[] a`) mit N Elementen (mehrere hunderttausend bis Millionen) erstellt und mit Zufallszahlen füllt.

B. Implementierung des Timers (Worst-Case-Fokus)

1. Implementieren Sie im Client einen automatischen Timer. Der Timer soll die Zeit messen, die für einen einzelnen Aufruf von `linearSearch` benötigt wird.
2. Provozieren Sie den Worst Case: Rufen Sie `linearSearch` so auf, dass der zu suchende Wert `val` garantiert nicht im Array `a` enthalten ist (oder das letzte Element ist). *Dieser Fall erzwingt die Überprüfung aller N Elemente.*
3. Ausgabe: Geben Sie die gestoppte Laufzeit (z.B. in Nanosekunden oder Millisekunden) über die Standardausgabe aus.

Teil 2: Messung, Visualisierung und Big-O-Prognose

Untersuchen Sie, wie sich die Laufzeit $f(n)$ verhält, wenn die Eingabegröße n zunimmt.

A. Messung des Skalierungsverhaltens

1. Wiederholen Sie die Worst-Case-Messung (gemäß Teil 1.B) für mindestens vier unterschiedliche, exponentiell wachsende Array-Längen N .
2. Verdoppeln Sie die Arraylänge N schrittweise (z.B. $N = 100.000, 200.000, 400.000, 800.000$).

B. Analyse und Prognose

1. Dokumentation und Visualisierung:
 - Dokumentieren Sie Ihre Ergebnisse, indem Sie die gemessenen Laufzeiten tabellarisch (Liste oder Tabelle) notieren: Eingabegröße N vs. Zeit $f(N)$.
 - Visualisieren Sie die gemessenen Laufzeiten in einem Koordinatensystem (z.B. mithilfe eines externen Tools oder handschriftlich): N auf der x-Achse, Zeit $f(N)$ auf der y-Achse.
2. Big-O Prognose:
 - Prognostizieren Sie anhand Ihrer gemessenen Daten und der Kurve das Laufzeitverhalten Ihres Algorithmus.
 - Erläutern Sie präzise, warum die lineare Suche im Worst Case als $O(n)$ (lineare Laufzeit) klassifiziert wird.
 - Begründen Sie Ihre Messungen:
 - Zeigen Sie mithilfe Ihrer Daten, dass eine Verdopplung der Eingabegröße N (Array-Länge) ungefähr zu einer Verdopplung der gemessenen Laufzeit führt.
 - Vergleichen Sie dieses empirische Ergebnis mit der formalen Big-O-Definition, die besagt, dass $f(n) \leq c \cdot g(n)$ gilt, wobei $g(n)=n$.